

Software Requirements Specification (SRS)

Project Name: StudyBearer **Version:** 1.0 **Target Event:** iSITE AI Hackathon 2026

1. Introduction

1.1 Purpose

This document specifies the software requirements for StudyBearer, an AI-powered educational workflow application. The purpose of this system is to automate the cognitive load of study planning and testing by transforming passive course materials into an active, scheduled, and gamified learning pipeline.

1.2 Scope

StudyBearer is a web-based application utilizing an Astro/React frontend and a Supabase backend, with core application logic and AI orchestration managed entirely by the n8n platform. The system will allow users to upload PDF materials, specify study duration, automatically generate and push a chunked study roadmap to Google Calendar, and administer AI-generated quizzes post-study. It includes a competitive progression system to incentivize consistency.

1.3 Definitions & Acronyms

- **PRD:** Product Requirements Document
 - **SRS:** Software Requirements Specification
 - **n8n:** A node-based workflow automation tool used as the central logic engine.
 - **RAG:** Retrieval-Augmented Generation (used for parsing PDF context).
 - **Elo:** A rating system used to calculate the relative skill levels of players, adapted here for academic consistency.
-

2. Overall Description

2.1 Product Perspective

StudyBearer is an independent web application that interfaces heavily with third-party APIs (Google Calendar, OpenAI/Gemini via n8n). It replaces traditional, manual study scheduling and static flashcards with an automated, push-based system.

2.2 User Classes and Characteristics

- **The Overwhelmed Student:** Users struggling with executive dysfunction, punctuality, or the sheer volume of heavy technical subjects (e.g., breaking down a massive Data Structures module). They need the system to take over the planning phase completely.
- **The Competitive Learner:** Users motivated by visual progression, leaderboards, and gaming mechanics. They respond well to maintaining combos, earning "First Blood" achievements, and climbing competitive ladders up to a "Mythic" tier.

2.3 Operating Environment

- **Frontend:** Modern web browsers (Chrome, Firefox, Safari, Edge) on desktop and mobile.
 - **Backend Infrastructure:** Supabase (PostgreSQL), Prisma ORM, Node.js environment.
 - **Orchestration:** n8n cloud or self-hosted instance for handling webhooks and AI agent routing.
-

3. System Features (Functional Requirements)

3.1 Authentication Module

- **REQ-1.1:** The system shall allow users to authenticate using Google OAuth via Supabase to streamline access to Google Calendar APIs.
- **REQ-1.2:** The system shall create a base user profile upon first login, initializing their `current_streak` to 0 and their base competitive tier.

3.2 Document Intake & Processing

- **REQ-2.1:** The system shall allow users to upload PDF documents (up to a defined file size limit) to Supabase storage.
- **REQ-2.2:** The user interface shall capture user parameters: desired total study hours, target completion date, and preferred study days.
- **REQ-2.3:** Upon upload, the system shall trigger an n8n webhook containing the file reference and user parameters.

3.3 n8n AI Orchestration & Calendar Sync (Core Logic)

- **REQ-3.1:** The n8n workflow shall extract raw text from the uploaded PDF.
- **REQ-3.2:** The n8n AI agent shall process the text and chunk the curriculum into a structured JSON array of daily learning objectives.

- **REQ-3.3:** For each chunked objective, the n8n workflow shall make an API call to the user's Google Calendar to create a timed event.
- **REQ-3.4:** The calendar event description shall contain the specific sub-topics to be studied during that block (e.g., "Today's Focus: Linked Lists vs. Arrays").

3.4 Gamified Assessment Engine

- **REQ-4.1:** The system shall unlock a specific quiz module in the frontend dashboard only after the corresponding Google Calendar event time has passed.
- **REQ-4.2:** Upon initiating a quiz, an n8n webhook shall generate a dynamic JSON payload containing:
 - Multiple Choice Questions (MCQ)
 - Enumeration Questions
 - Identification Questions
- **REQ-4.3:** The React frontend shall render these questions interactively, providing immediate feedback upon submission.

3.5 Progression & Gamification Logic

- **REQ-5.1:** The system shall calculate a daily streak. Completing the scheduled study and passing the quiz adds +1 to the current_streak.
- **REQ-5.2:** Failing to complete a scheduled module within a 24-hour window shall reset the current_streak to 0.
- **REQ-5.3:** The system shall award Elo/points based on quiz accuracy and current streak multipliers.
- **REQ-5.4:** The system shall feature tiered ranks (e.g., Warrior, Elite, Master, Grandmaster, Epic, Legend, Mythic) that update automatically based on accumulated Elo.
- **REQ-5.5:** The system shall unlock specific cosmetic achievements (e.g., "First Blood" for the first quiz, "Flawless Victory" for a 100% score) managed via the UserAchievements relational table.

4. External Interface Requirements

4.1 User Interfaces

- **REQ-UI-1:** The application shall utilize an overarching dark-toned, cinematic, and moody visual aesthetic. Deep blacks, high-contrast typography, and subtle glowing neon accents will be used to simulate a premium gaming interface.
- **REQ-UI-2:** The dashboard shall display a visual roadmap of the chunked PDF, visually distinguishing locked, current, and completed modules.
- **REQ-UI-3:** Achievement unlocks and rank-ups shall trigger full-screen, high-fidelity CSS/React-spring animations.

4.2 Software Interfaces

- **Google OAuth 2.0 & Calendar API:** For user authentication and writing study blocks directly to the user's personal schedule.
 - **Supabase (PostgreSQL & Storage):** Used as the primary database and file repository.
 - **n8n Webhooks:** The primary communication bridge between the Astro frontend, the Supabase database, and the AI models.
 - **OpenAI/Gemini API:** Accessed exclusively through n8n nodes for RAG processing, curriculum design, and quiz generation.
-

5. Non-Functional Requirements

5.1 Performance Requirements

- **REQ-NF-1:** The Astro "Islands" architecture shall ensure the initial page load time is under 1.5 seconds by serving static HTML and only hydrating the React quiz/dashboard components when necessary.
- **REQ-NF-2:** The n8n processing pipeline for PDF chunking and Calendar sync shall complete within 30 seconds of file upload. The UI must provide a cinematic loading state during this orchestration.

5.2 Security Requirements

- **REQ-NF-3:** PDF files stored in Supabase shall be restricted via Row Level Security (RLS) policies so they are only accessible to the uploading user.
- **REQ-NF-4:** Google Calendar API tokens must be handled securely within the Supabase auth instance and never exposed to the client-side browser.

5.3 Reliability & Availability

- **REQ-NF-5:** The n8n workflows must include error-handling nodes (e.g., Catch nodes) to prevent silent failures if the AI model hallucinates malformed JSON or if the Google Calendar API rate limits the requests.

5.4 Maintainability

- **REQ-NF-6:** Database interactions shall be strictly type-checked using Prisma Client to ensure rapid, error-free schema updates during the tight time constraints of the hackathon.